

Practica de clase 3

Este es el laboratorio práctico para la comprensión de cómo afecta la organización de un computador en la forma de ser programado de forma óptima.

Para este laboratorio, necesitará:

- Una computadora con procesador x86-64.
- Sistema Operativo Linux con su entorno de desarrollo.
- Los códigos del TecDigital

Este laboratorio está diseñado para ejecutarse de forma individual.

Objetivo

Este laboratorio tiene como objetivo comprender, mediante un ejercicio práctico, los efectos de utilizar alineamiento, multihilos y afinidad.

Preparación del entorno

En el sistema de desarrollo (computadora personal o sistema empujado), verifique que tiene instalado el compilador de C:

```
gcc --version
```

Si el comando no existe:

```
sudo apt update  
sudo apt install build-essential
```

Para todos los ejercicios, debe usar *intrinsics* de AVX2. Puede utilizar la plantilla proporcionada en el TecDigital como punto de partida.

Ejercicio A

Compile y ejecute el código de `cpu-affinity` y `cpu-naive`. Por ejemplo:

```
time ./cpu-affinity
```

El resultado del comando son distintos tiempos. Tome nota del tiempo *real*.

Luego modifique el número de hilos desde 1 hasta el máximo disponible en su computadora. Luego realice un gráfico con los tiempos *real*. Luego, explique con lo visto en clase y sobre evaluación de rendimiento cómo es la escalabilidad, de cuánto es la proporción de código paralelo con respecto al total y la eficiencia.

Ejercicio B

Compile y ejecute el código de `matmul_tiled.openmp` y `softmax.openmp`.

Luego modifique el número de hilos desde 1 hasta el máximo disponible en su computadora, a través del argumento de la aplicación:

```
./softmax.openmp 2
```

donde 2 es el número de hilos.

Luego realice un gráfico con los tiempos reportados por la aplicación. Luego, explique con lo visto en clase y sobre evaluación de rendimiento cómo es la escalabilidad, de cuánto es la proporción de código paralelo con respecto al total y la eficiencia.

Ejercicio C

Siguiendo el código realizado en el Laboratorio 2 del Capítulo 1, vectorice el código de ambos programas y repita las mediciones, como hizo en los ejercicios A y B.

Entregable

Es un ejercicio para realizar en clase. Debe incluirse dentro de un repositorio de Git donde se manejen los laboratorios de clase. Se debe subir el enlace del repositorio a través del TecDigital.

Practica de clase 4

Este es el laboratorio práctico para la comprensión de bibliotecas estáticas, bibliotecas dinámicas e *inlining* en C.

Para este laboratorio, necesitará:

- Una computadora con procesador x86-64.
- Sistema Operativo Linux con su entorno de desarrollo.
- Compilador gcc.
- Los códigos del TecDigital.

Este laboratorio está diseñado para ejecutarse de forma individual.

Objetivo

Este laboratorio tiene como objetivo comparar el desempeño y el tamaño de archivos generados al utilizar bibliotecas estáticas, bibliotecas dinámicas y funciones declaradas como `static inline`.

Preparación del entorno

Verifique que tiene instalado el compilador de C:

```
gcc --version
```

Si el comando no existe:

```
sudo apt update  
sudo apt install build-essential
```

Luego compile todos los ejemplos:

```
make -C libraries all
```

Ejercicio A

Ejecute la versión enlazada con biblioteca estática:

```
./libraries/build/bin/bench-static 1000000 1000 1.0 2.0
```

Anote los tiempos de llenado y suma. Luego identifique el archivo de biblioteca estática generado:

```
ls -lh libraries/build/lib/libvectorops.a
```

Ejercicio B

Ejecute la versión enlazada con biblioteca dinámica:

```
./libraries/build/bin/bench-dynamic 1000000 1000 1.0 2.0
```

Anote los tiempos de llenado y suma. Luego identifique el archivo de biblioteca dinámica generado:

```
ls -lh libraries/build/lib/libvectorops.so
```

Compare el resultado contra la versión estática.

Ejercicio C

Ejecute la versión con funciones `static inline`:

```
./libraries/build/bin/bench-inline 1000000 1000 1.0 2.0
```

Anote los tiempos de llenado y suma. Luego explique por qué el compilador puede optimizar esta versión de forma distinta a las versiones con biblioteca.

Ejercicio D

Compare el tamaño de los ejecutables y bibliotecas:

```
make -C libraries size
```

Anote el tamaño de:

- `bench-static`
- `bench-dynamic`
- `bench-inline`
- `libvectorops.a`
- `libvectorops.so`

Luego explique cuál ejecutable es más grande y cuál depende de un archivo externo para ejecutarse.

Ejercicio E

Ejecute todas las versiones:

```
make -C libraries run-all ARGS="1000000 1000 1.0 2.0"
```

Incluya también la versión `bench-static-lto`. Compare los tiempos contra la versión `static inline` y explique si `flto` permite al compilador optimizar entre archivos.

Entregable

Es un ejercicio para realizar en clase. Debe incluirse dentro de un repositorio de Git donde se manejen los laboratorios de clase. Se debe subir el enlace del repositorio a través del TecDigital.

El repositorio debe incluir:

- Los códigos utilizados.
- El `Makefile`.
- Una tabla con los tiempos medidos y un análisis de por qué se obtienen los resultados.
- Una tabla con el tamaño de los archivos generados y un análisis de por qué se obtienen los resultados.
- Una breve explicación de las diferencias entre biblioteca estática, biblioteca dinámica e *inlining*.

Practica de clase 5

Este es el laboratorio práctico para la comprensión de ensamblador incrustado en C y el efecto de distintas optimizaciones comunes del compilador.

Para este laboratorio, necesitará:

- Una computadora con procesador x86-64.
- Sistema Operativo Linux con su entorno de desarrollo.
- Compilador gcc.
- Los códigos del TecDigital.

Este laboratorio está diseñado para ejecutarse de forma individual.

Objetivo

Este laboratorio tiene como objetivo comparar, mediante ejercicios prácticos, una implementación en C contra una implementación con ensamblador incrustado, así como analizar el efecto de distintas banderas de optimización del compilador.

Preparación del entorno

En el sistema de desarrollo, verifique que tiene instalado el compilador de C:

```
gcc --version
```

Si el comando no existe:

```
sudo apt update  
sudo apt install build-essential
```

Luego verifique que puede compilar los ejemplos:

```
make -C asm-c all  
make -C optimisations all
```

Ejercicio A

Compile y ejecute el código de `popcount-naive.c`:

```
make -C asm-c run
```

El programa compara una función en C para contar bits encendidos contra una función que utiliza ensamblador incrustado con la instrucción `popcnt`.

Anote los tiempos reportados por la aplicación y el *speedup*. Luego explique por qué la instrucción `popcnt` puede ser más rápida que una solución iterativa en C.

Ejercicio B

Compile el mismo programa con las distintas banderas de optimización incluidas en el **Makefile**:

```
make -C asm-c run-all
```

Tome nota de los tiempos para al menos las versiones 00, 02, 03, **Ofast**, **march-native** y **lto**. Luego responda:

- ¿Cuál versión fue más rápida?
- ¿Todas las optimizaciones mejoraron el tiempo?
- ¿El ensamblador incrustado cambió su ventaja con las distintas banderas?

Ejercicio C

Compile y ejecute el código de suma de vectores:

```
make -C optimisations run
```

El programa reserva memoria para tres vectores de **double**, llena dos vectores de forma iterativa y luego realiza la suma elemento por elemento. La aplicación reporta el tiempo de asignación, llenado, suma y liberación de memoria en microsegundos.

Ejecute el programa con al menos tres tamaños distintos:

```
./optimisations/build/vector-add-02 10000 1.0 2.0 1000  
./optimisations/build/vector-add-02 100000 1.0 2.0 1000  
./optimisations/build/vector-add-02 1000000 1.0 2.0 1000
```

Anote los tiempos de llenado y suma. Luego explique cómo cambia el tiempo al aumentar el tamaño de los vectores.

Ejercicio D

Ejecute todas las versiones optimizadas de la suma de vectores:

```
make -C optimisations run-all ARGS="1000000 1.0 2.0 1000"
```

Compare los tiempos de **fill A**, **fill B** y **add vectors** para las distintas versiones. Incluya al menos las versiones 00, 02, 03, **Ofast**, **march-native**, **vectorize** y **no-vectorize**.

Luego genere el reporte de vectorización:

```
make -C optimisations vector-report
```

Explique si el compilador logró vectorizar los ciclos principales y relacione esa información con los tiempos observados.

Ejercicio E

Compile la versión con optimización guiada por perfil:

```
make -C optimisations pgo PGO_ARGS="1000000 1.0 2.0 1000"  
./optimisations/build/vector-add-pgo 1000000 1.0 2.0 1000
```

Compare el resultado contra 02, 03 y `march-native`. Luego indique si la optimización guiada por perfil produjo una mejora visible en este caso.

Entregable

Es un ejercicio para realizar en clase. Debe incluirse dentro de un repositorio de Git donde se manejen los laboratorios de clase. Se debe subir el enlace del repositorio a través del TecDigital.

El repositorio debe incluir:

- Los códigos utilizados.
- Los `Makefile`.
- Una tabla con los tiempos medidos.
- Una breve explicación de los resultados de cada ejercicio.